

Compute Systems Engine: 代码 实践卷

从 reference 到可恢复分布式计算引擎

CPU Performance Study

本卷是《从 C++ 到计算系统：第二册》的配套代码实践卷，专门承接完整 C++20 实现、测试、benchmark、故障注入和实验报告。

前言

第二册原理卷负责回答“为什么”：为什么热循环会被分支、依赖和内存等待限制，为什么 false sharing 会让正确程序扩展失败，为什么条件变量必须围绕等待谓词，为什么异步 I/O 的 submit、completion 和 commit 不是同一件事，为什么分布式系统必须区分逻辑任务和物理 attempt。本卷负责回答“怎样落地”：怎样把这些原则写成一个可以构建、测试、测量、故障注入和复盘的 C++20 工程。

本卷的贯穿项目统一称为 **Compute Systems Engine**（计算系统引擎）。它从一个日志统计任务开始：输入是一批记录，输出是按 key 聚合的稳定结果。第一阶段只要求顺序 reference、输入合同和诊断；后续逐步加入 split、batch、冷热字段、benchmark、并行内核、线程池、有界队列、原子协议、任务图、可靠写、manifest、checkpoint、MessageBus、attempt、epoch、分片和故障注入。每一步都必须有测试和报告，不接受只写代码、不解释边界的实现。

本卷不是代码片段合集，而是一部工程教材。每章都遵循同一条路线：先说明本章要兑现原理卷中的哪个机制；再给出接口和目录组织；随后实现最小版本、测试失败输入、补充报告字段；最后说明这个版本还不能处理什么，下一章为什么必须继续扩展。读者应当把代码、测试和报告看成一个整体：代码实现机制，测试固定语义，报告保存证据。

核心思想

代码实践卷的目标不是写一个庞大的教学框架，而是让每一项复杂度都有购买理由：它修复了哪个失败，保留了哪个正确性边界，带来了什么运行成本，怎样用测试和报告证明。

全书结构

本卷按 Compute Systems Engine 的工程演化组织。第一部分建立输入合同、reference、诊断和测试框架。第二部分加入数据布局、热路径和 benchmark harness。第三部分加入并行内核、线程池、同步和原子协议。第四部分加入任务图、运行时、异步 I/O、可靠写和 checkpoint。第五部分加入 MessageBus、attempt、epoch、分片、复制和端到端故障注入。

每个阶段都必须产出四类东西。第一，稳定接口，例如 `InputSplit`、`RecordId`、`HotBatch`、`QueueContract`、`ManifestEntry` 或 `RunReport`。第二，可运行实现，至少包含 reference 和一个带机制的版本。第三，测试，包括正常输入、边界输入和能击穿朴素方案的失败输入。第四，报告，记录正确性、性能、资源、故障和未验证边界。

本卷与第二册原理卷并行阅读。原理卷先解释机制为什么存在，本卷再把机制落成代码。若读者只读本卷而跳过原理卷，容易把接口当成固定答案；若只读原理卷而不做本卷，容易把工程边界停留在口头推导。两卷合起来，才形成从零到精通的完整训练。

目录

第一部分	从 reference 到可测试工程	
第 1 章	输入合同、Reference 与第一组测试	2

第零部分

从 reference 到可测试工程

第 1 章

输入合同、Reference 与第一组测试

1.1 本章目标

本章实现 Compute Systems Engine 的第一个可运行切片：输入合同、顺序 reference、诊断结果、稳定输出和测试。它对应原理卷第一章的内容。此时不做并行、不做 SIMD、不做 manifest，也不追求吞吐；目标只有一个：让项目拥有一把可信尺子。后续所有优化都必须先通过这把尺子。第一版工程应至少包含四个模块：

模块	职责	不负责
input	读取固定测试输入，分配 RecordId	并行切分和异步 I/O
reference	顺序解析和计数	优化、缓存布局和线程
report	输出 accepted、rejected、counts 和 checksum	性能归因
tests	固定正常输入、坏行和稳定输出	压力 benchmark

1.2 第一版接口

第一版接口要小，但不能含糊。建议先定义四个概念：RecordId、ParseDiagnostic、ReferenceResult 和 InputCase。完整字段可以随项目扩展，但第一版必须能回答三件事：哪条记录被接受，哪条记录被拒绝，最终计数是否稳定。

Listing 1.1 第一版核心结构

```
1 struct RecordId {
2     std::string input_name;
3     std::uint64_t input_generation = 0;
4     std::uint64_t byte_offset = 0;
5     std::uint64_t line_number = 0;
6 };
7
8 struct ParseDiagnostic {
9     RecordId record;
10    std::string error_kind;
11 };
12
13 struct ReferenceResult {
14     std::uint64_t accepted = 0;
15     std::vector<ParseDiagnostic> diagnostics;
16     std::map<std::string, std::uint64_t> counts;
17 };
```

1.3 测试先于优化

第一组测试不追求覆盖所有未来功能，只固定最核心语义。正常五行输入应得到 view=2；缺字段输入应进入 diagnostics；字段过多、空事件名和 value 非整数都应有明确错误类型；输出 key 顺序必须稳定。若这些测试不稳定，后续任何性能实验都应暂停。

习题与作业

本章实现任务：建立 CMake target、顺序 reference 和测试。测试至少包含 normal five lines、missing field、extra field、empty event name、bad value 五类输入。每个测试同时检查 accepted、diagnostics、counts 和稳定 checksum。

1.4 报告格式

第一版报告可以是简单 key-value 文本，不需要复杂 UI。最低字段包括输入名、输入版本、总行数、accepted、rejected、输出 checksum 和错误类型计数。报告不是给人截图看的，而是后续脚本和读者复查用的证据。

Listing 1.2 第一版报告示例

```
input_name=normal_5_lines
input_generation=1
records_total=5
accepted=5
rejected=0
checksum=...
count.click=1
count.login=1
count.logout=1
count.view=2
```

本章结束时，项目还很小，但它已经具备工程地基：语义明确、测试可跑、报告可查。下一章才能开始讨论 split 和 RecordId 在并行输入中的作用。

术语表

Compute Systems Engine 本卷贯穿项目，用于承接第二册原理卷中的硬件、并发、运行时、I/O 和分布式计算机制。

reference 参考实现，用清楚、确定、可复查的方式定义正确结果。

oracle 判定器，用于比较 reference 和待测实现的输出是否满足同一语义。

索引

Compute Systems Engine, ii